

Character System

Character Profile (Static/Semi-Static Data)

- **Demographics**
 - Background (string) ex , “They dated in middle school, broke up and got back together in university”
 - Name
 - Age
 - Appearance description
 - Race
 - Gender
 - Occupation
 - Etc.
- During Generation
 - 1st shot: create background strings of all characters
 - 2nd shot: parse/populate all relationships and shit

Character State

- **Personality Traits**
 - Ambition level
 - Confrontational tendency
 - Core motivations
 - ...
- **Needs/Status (0-100 scales)**
 - Happiness
 - Energy
 - Hunger
 - Hygiene
 - ...
- **Current State**
 - Current desire/intention. Ex. Wants to go swim. Wants to visit a friend. Wants to...

Character Memory

- **Action Log** - Timestamped record of actions taken
- **Memory Log** - Significant events/observations from space contexts
- **Relationships Map**

Character Methods

.decide() → Makes an LLM call to change desire/immediate next action based on context
.move(location) → Navigate to coordinates/space

.initiateAction(target, actionType) → Start interaction
We talked about it as .dialog, or .fight, etc (change this if you want)
.use(object) → Generic object interaction, dialog box with text

Relationship System

Data Structure

- **Format:** Adjacency list
- Created automatically when two characters first interact

Relationship Object

```
Relationship {  
  - targetCharacter: Character  
  - relationshipType: enum (Friendly, Romantic, Antagonistic, Professional, Family)  
  - relationshipSummary: string (LLM-generated narrative)  
  - relationshipScore: int (-100 to +100)  
  - interactionHistory: List<InteractionSummary>  
  - currentInteractionState: InteractionState ex. Kissing, talking, fucking, fighting, null  
}
```

Interaction Flow

1. **During Interaction:** Full conversation/context maintained in memory
 2. **After Interaction:** LLM generates summary → stored in interactionHistory
 3. **Summary includes:** What happened, emotional impact, relationship changes
-

Space System

Space Definition

- Ex. Arihan's House, Town Square, Basketball Court
- **Static Properties**
 - Name
 - Boundaries (hardcoded areas)
 - Available objects/interactables
- **Dynamic State**
 - Characters currently present
 - Ongoing activities (LLM-generated description)

Space State Management

- **Update Trigger:** When character performs action in space
 - **Process:** LLM summarizes current activities of all characters in space
 - **Example State:** "Sarah is showering. Arihan is cooking dinner. Music playing softly."
-

Context System

Global Context (Shared by all)

- Complete map layout
- All character positions
- Time of day / weather

Local Context (Per Character)

- **Perception Radius:** Circular AOE around character
- **Contains:**
 - Space states for any intersecting spaces + objects in those spaces

Context Update Triggers

- Character moves to new space.
 - Ex. We pass the school area - gaining context of my son fighting another kid - We take immediate action to go talk to son
 - Event occurs in perceived space.
 - Ex. Kid walks into lumber yard
 - → Triggers `.decide()` re-evaluation
-

Action/Interaction System

Action Types

- **Object Interactions:** `.use(object)` - Generic LLM-generated flavor text
- **Character Interactions:** Turn-based with specific types:
 - `.dialog` - Conversation exchange
 - `.fight` - Physical confrontation
 - `.romance` - Romantic interaction
 - ...

Interaction Mechanics

1. **Initiation:** Character A initiates with Character B

2. **State Lock:** Both characters' `relationship` mapping `interactionState` set
3. **Turn-Based Exchange:**
 - Full conversation context maintained
 - Actions affect parametric values (e.g., insult → anger ↑)
4. **Sub-Actions:**
 - `.continue()`
 - `.leave()`
 - `.talk()`
 - ...
5. **Termination:**
 - LLM generates interaction summary
 - Summary stored in both characters' relationship histories
 - States reset

Character Background:

Background String

Demographic

- Name
- Age
- Appearance
- Race
- Gender
- Occupation
- ...

Character Context/State:

Relationships

- Adjacency list of people
- Any interactions with another character creates an entry
- Within each relationship:
 - Relationship Background (String):
 - Describe the relationship in words
 - "They dated in middle school, broke up and got back together in university"
 - Relationship Category, ex. Loving, Antagonistic
 - interactions
 - During the conversation, the characters have full context (entire chat/context dump)
 - After conversation ends, we make an AI summary and store that
 - Interaction state
 - Enum: Kissing, talking, fucking, fighting, FALSE

Characters

- Action Log
 - Log of actions taken
 - Past actions will be used to infer future actions/desires
- Memory Log
 - Log of things from stored space states/contexts
- Personality Traits
 - Ex. Are they ambitious? Are they confrontational? What do they want to do?
- Parametric values, state values
 - Happiness
 - Sadness
 - Hunger
 - Shitiness

- Desire/Immediate next action
 - Ex. Wants to go swim. Wants to visit a friend. Wants to...
- Is interacting
- Functions:
 - .decide
 - Makes an LLM call to change desire/immediate next action based on context
 - .move?
 - Move to a particular location
 - Actions
 - When close to someone, set their relationship interaction state(two way) to whatever the action is
 - Change parametric values of people. Ex. I call arihan a bitch, anger goes up
 - When an action ends, summarize the action and put it in the person's relationship's interactions
 - Action types:
 - .talk
 - .fight
 - .wtv
 - Actions are a turn based "interaction"
 - Functions within an interaction
 - .leave
 - .talk
 - Etc. Defined per action class
 - .use
 - Special Action for all objects that just generates generic text to place above a person while interacting with objects.

Spaces

- Spaces are hardcoded areas boxed off on the map
 - Ex. Arihan's House
 - Town Square
 - Basketball Court
- Spaces have their state internally stored/contexts
 - Map over people in a space and make an LLM call to update the state based on the people's actions
 - Sarah is taking a shower
 - Aadi is gooning
 - Arihan is cooking

Contexts

Each character has a local and global context. Everyone shares a global context

- Global Context

- Map
- Positions of characters
- Local Context
 - When a character's "circle/AOE" intersects with a space, the character gains context of that space's stored state

Character Actions/Interactions:

- When: Every time a character's local context changes, .decide function is called
 - Ex. We pass the school area - gaining context of my son fighting another kid - We take immediate action to go talk to son
 - Ex. Kid's fight comes into lumber yard
- Interactions
 - Any character can initiate an action with another character OR object at any time that is within their local context

